PROGRAMMING LANGUAGES II (JAVA)

Student Budget Management System

Course: COM 141: Programming Languages II (Java)

Student: Aizhan Esenbekova

Group: COMFCI-24

Program: Console Budget Manager

Goal of the project

- Create a simple console system to manage personal finances
- Track accounts, expenses, and savings goals
- Apply data structures effectively
- Analyze resulting algorithm complexity

Problem & Constraints

The Problem

Students need a simple, lightweight tool to manage multiple financial accounts, track spending by precise categories, save money efficiently for goals, and prevent unauthorized operations on locked accounts.

Functional Requirements

- Create, deposit, and withdraw funds
- Transfer money between accounts
- Record and analyze category spending
- Set and monitor savings goals
- Secure accounts with lock/unlock actions

Uses only standard libraries without crashing

Problem & Constraints

System Constraints

- Commands are **case-insensitive (OPEN = open)**, while account/goal identifiers maintain strict case-sensitivity.
- All money **inputs must be integers**. Zero or negative entries are rejected via try-catch.
- **Strict balance checks:** Withdraw, Spend, and Save cannot exceed current available funds.
- **Critical Logic:** The SAVE command strictly rejects targets if the goal hasn't been pre-defined via SET_GOAL to ensure data consistency.

System Requirements

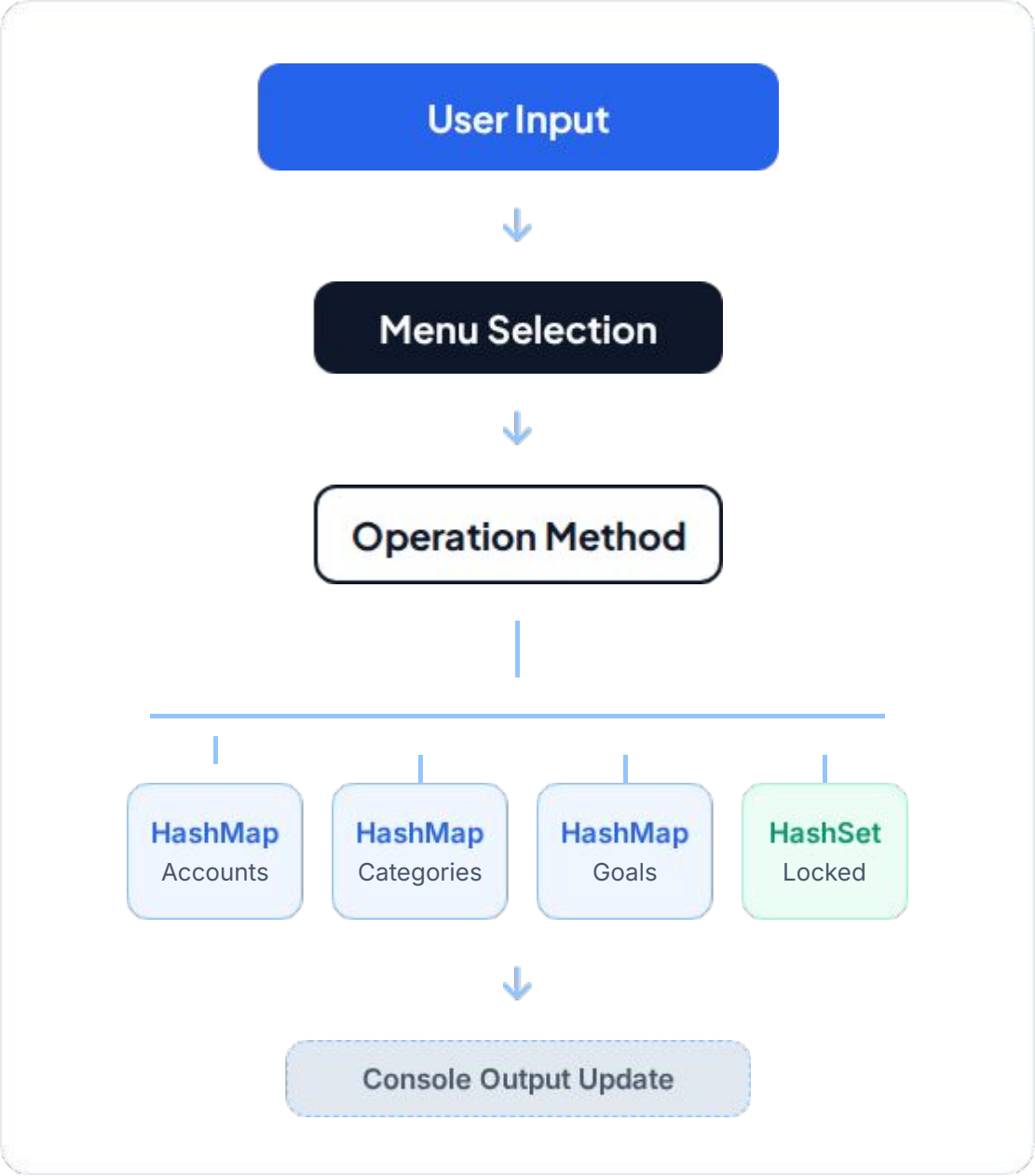
- **Single-File Architecture:** Entire logic contained in one .java file using static methods only.
- **No Custom Classes:** Rely exclusively on standard Java primitives and Collections.
- **Monetary Consistency:** All financial values handled as integers in **Kyrgyz Soms (KGS)**.
- **No Filesystem I/O:** Purely in-memory state management with console interactions.

Data Structures Used

The program uses two core structures from the **Java Collections Framework** to ensure optimal performance and code simplicity.

- **HashMap:** For key-value financial associations.
- **HashSet:** For unique status tracking.

Why: These structures provide **O(1) average access**, which is critical for maintaining an instantaneous user experience even as account data scales.



HashMap: Accounts & Spending

1. HashMap (Balances)

HashMap balanceByAccount

Stores account names as keys and balances as values. Provides extremely fast lookups.

```
Account | Balance
-----------------
Aida | 500
Bek | 1200
```

- **Fast Lookup:** Access by account name.
- **Uniqueness:** Names act as unique keys. Direct access using account name
- **O(1):** Constant time for get/put. Efficient updates when depositing or withdrawing

Handles deposits, withdrawals, and transfers instantly.

2. HashMap (Categories)

HashMap categorySpend

Tracks total spending dynamically mapped to string-based category names.

```
Category | Total
-----------------
Food | 300
Transport | 100
```

- **Classification:** Maps spending to tags. Each category is a unique key
- **Easy Updates:** Updates totals via keys. Fast updates when recording expenses
- **Fast Stats:** Quick access to totals. Allows quick reporting of spending statistics

Example: Food → 350, Transport → 120.

HashMap Goals & HashSet Locks

3. HashMap (Dual Goals)

Two separate maps handle goal tracking cleanly without complex nested objects.

```
goalTarget: Laptop -> 2000
goalSaved: Laptop -> 500
```

- **Tracking:** Pairs targets with progress + Goals are uniquely identified by name
- **Incremental:** Fast saving updates.
- **O(1):** Efficient status checking.

Supports "Laptop → 2000" and progress tracking.

4. HashSet (Locks)

Stores unique strings of blocked accounts. Ensures absolute uniqueness and constant-time lock verification.

```
[ "Aida", "Bek" ]
```

- **Duplicates:** Prevents double-blocking.
- **Security:** Fast membership check.
- **O(1):** Constant contains() check.

System verifies locks before any withdrawal action.

Data Structure Summary

Structure	Component	Primary Purpose	Advantage
HashMap	Accounts	Balance storage/lookup	Instant access by ID
HashMap	Categories	Total spending analysis	Easy tag updates
HashMap	Goals	Milestone tracking	Fast progress updates
HashSet	Locked	Membership verification	Unique, fast lookup

Conclusion: High scalability and O(1) efficiency across all core operations.

>_ Demo: Console I/O Executions

```
> HELP

> OPEN Card
> OPEN Cash

> DEPOSIT Card +5000
> DEPOSIT Cash +1000

> BALANCE Card
5000

> TRANSFER
Card —> Cash 1000

> BALANCE Cash
2000

> ACCOUNTS
Card : 4000
Cash : 2000
```

```
Commands:
HELP
OPEN
DEPOSIT
WITHDRAW
TRANSFER
BALANCE
ACCOUNTS
SPEND
CATEGORY
TOP_CATEGORIES
SET_GOAL
SAVE GOAL_STATUS
GOALS
LOCK UNLOCK
LEDGER
RESET
SET_BUDGET
BUDGET_STATUS
RULE_SAVE
APPLY_RULES
REPORT
EXIT
Command:
```

```
Command: OPEN
Account: CARD
Account created: 🥳
```

```
Command: OPEN
Account: CASH
Account created: 🥳
```

```
Command: DEPOSIT
Account: CARD
Amount: 5000
Deposit complete 💚
```

```
Command: DEPOSIT
Account: CASH
Amount: 1000
Deposit complete 💚
```

```
Command: BALANCE
Account: CARD
Balance: 5000
```

```
Command: TRANSFER
From: CARD
To: CASH
Amount: 1000
Transfer complete 👍
```

```
Command: BALANCE
Account: CASH
Balance: 2000
```

```
Command: ACCOUNTS
CASH : 2000
CARD : 4000
```

> **SPEND** Cash food 300

```
Command: SPEND
Account: CASH
Category: FOOD
Amount: 300
Withdraw complete💚
```

> **SPEND** Cash transport 200

```
Command: SPEND
Account: CASH
Category: TRANSPORT
Amount: 200
Withdraw complete💚
```

> **CATEGORY** food 300

```
Command: CATEGORY
Category: FOOD
FOOD spent: 300
```

> **CATEGORY** transport 200

```
Command: CATEGORY
Category: TRANSPORT
TRANSPORT spent: 200
```

> **SET_BUDGET** food 250

```
Command: SET_BUDGET
Category: TRANSPORT
Amount: 250
```

> **SPEND** Cash food 100

```
Command: SPEND
Account: CASH
Category: FOOD
Amount: 100
Withdraw complete💚
ALERT: budget exceeded for FOOD
```

ALERT: budget exceeded for food

> **TOP_CATEGORIES 3**
1) food: 400
2) transport: 200

```
Command: TOP_CATEGORIES
k: 3
FOOD : 500
TRANSPORT : 200
```

> **SET_GOAL** Laptop 40000

```
Command: SET_GOAL
Goal: LAPTOP
Target: 40000
```

> **SAVE** Card Laptop 2000

```
Command: SAVE
Account: CARD
Goal: LAPTOP
Amount: 2000
```

> **GOAL_STATUS** Laptop
Target=40000, Saved=2000, Progress=5%

```
Command: GOAL_STATUS
Goal: LAPTOP
Saved: 2000 Target: 40000 Progress: 5%
```

> **GOALS**
Laptop 2000/40000

```
Command: GOALS
LAPTOP 2000/40000
```

Demo: Console I/O Executions

```
> LOCK Card

> WITHDRAW Card 500
Account locked

> UNLOCK Card
> WITHDRAW Card 500

> RULE_SAVE 10
> APPLY_RULES

> LEDGER

> REPORT

> RESET

> ACCOUNTS
(no accounts)

> EXIT
```

```
Command: LOCK
Account: CARD
```

```
Command: WITHDRAW
Account: CARD
Amount: 500
Account locked 🚫
```

```
Command: UNLOCK
Account: CARD
```

```
Command: WITHDRAW
Account: CARD
Amount: 500
Withdraw complete 💚
```

```
Command: RULE_SAVE
Percent: 10
Rule set
```

```
Command: APPLY_RULES
```

```
Command: LEDGER
OPEN CARD
OPEN CASH
DEPOSIT CARD 5000
DEPOSIT CASH 1000
TRANSFER CARD CASH 1000
WITHDRAW CASH 300
SPEND CASH FOOD 300
WITHDRAW CASH 200
SPEND CASH TRANSPORT 200
WITHDRAW CASH 100
SPEND CASH FOOD 100
WITHDRAW CASH 100
SPEND CASH FOOD 100
SET_GOAL LAPTOP
WITHDRAW CARD 2000
SAVE CARD LAPTOP 2000
WITHDRAW CARD 500
```

```
Command: REPORT

=== REPORT ===
CASH : 1170
CARD : 1350

Goals:
LAPTOP 2000/40000

Top categories:
FOOD : 500
TRANSPORT : 200
```

```
Command: RESET
System reset.💭
```

```
Command: ACCOUNTS
```

```
Command: EXIT
Bye.
```

⏱️ Empirical Timing Observation: Lock / Unlock Complexity

What We Tested

We performed a **small empirical timing experiment** in Java to observe the performance of the lock() and unlock() operations in our BudgetSystem.

The experiment measures the time required to repeatedly **lock and unlock accounts** using a HashSet that stores locked account IDs.

The timing was measured using: **System.nanoTime()**

This method provides **high-resolution timing in nanoseconds**, allowing us to estimate how long the operations take.

Experiment Setup

1. We created **N accounts** with initial balance 1000.
2. Accounts were stored in: **HashMap<String, Integer> balanceByAccount**
3. Locked accounts were tracked using: **HashSet<String> lockedAccounts**
4. For each account we executed: **lock(account)** AND **unlock(account)**
5. The experiment was repeated **7 times** (REPEATS = 7) to reduce random timing noise.
6. We tested three different input sizes:

```java
static void testLockUnlock(int N) {  3 usages  new *

    long total = 0;

    // prepare accounts
    BudgetSystem.balanceByAccount.clear();
    BudgetSystem.lockedAccounts.clear();

    for (int i = 0; i < N; i++) {
        BudgetSystem.balanceByAccount.put("A" + i, 1000);
    }

    for (int r = 0; r < REPEATS; r++) {

        long start = System.nanoTime();

        for (int i = 0; i < N; i++) {
            BudgetSystem.lock( acc: "A" + i);
            BudgetSystem.unlock( acc: "A" + i);
        }

        long end = System.nanoTime();

        total += (end - start);
```

```
Size: 10 → lock/unlock avg time (HashSet O(1) avg): 53142 ns
Size: 1000 → lock/unlock avg time (HashSet O(1) avg): 885042 ns
Size: 1000000 → lock/unlock avg time (HashSet O(1) avg): 167755200 ns
```

Analysis of Results

This indicates that the total runtime grows roughly **linearly with N** because the program performs **N lock/unlock operations**. However, each individual operation remains **constant time O(1)** on average because HashSet operations such as add() and remove() are O(1).

Empirical Timing: TOP_CATEGORIES

What We Tested

We performed a **small empirical timing experiment** to observe the performance of the **topCategories()**. The topCategories() method works by sorting the categories according to their spending amounts using **nested loops (selection sort)**.

```
Category9 : 9 KGS
Category8 : 8 KGS
Category7 : 7 KGS
Category6 : 6 KGS
Category5 : 5 KGS
Size: 10 → 2671500 ns
Category999 : 999 KGS
Category998 : 998 KGS
Category997 : 997 KGS
Category996 : 996 KGS
Category995 : 995 KGS
Size: 1000 → 34185300 ns
Category9999 : 9999 KGS
Category9998 : 9998 KGS
Category9997 : 9997 KGS
Category9996 : 9996 KGS
Category9995 : 9995 KGS
Size: 10000 → 772888100 ns
```

```java
static void testTopCategories(int N) {  3 usages  new *

    long total = 0;

    // prepare data
    BudgetSystem.categorySpend.clear();

    for (int i = 0; i < N; i++) {
        BudgetSystem.categorySpend.put("Category" + i, i * 10);
    }

    for (int r = 0; r < REPEATS; r++) {

        long start = System.nanoTime();

        BudgetSystem.topCategories( k: 5);

        long end = System.nanoTime();

        total += (end - start);
```

Analysis of Results

The results show that the execution time **increases rapidly as the number of categories grows**. This happens because the topCategories() method uses **nested loops to sort categories**, which results in a **quadratic time complexity**: **$O(n^2)$**

When the number of categories increases, the number of comparisons grows approximately proportional to **n^2**, causing the runtime to increase significantly.

From this experiment, we observed that the **topCategories() operation becomes significantly slower as the number of categories increases**.

This occurs because the algorithm relies on **nested loops for sorting**, resulting in **$O(n^2)$ time complexity**.The empirical timing measurements confirm that operations with quadratic complexity scale poorly for large input sizes compared to constant-time operations such as those based on **HashSet or HashMap**.

Experiment Setup

1. We created **N categories** with different spending values.
2. Categories were stored in: **HashMap<String, Integer> categorySpend**
3. Each category had a spending value such as:

   Category0 → 0 KGS and **Category1 → 1 KGS**

4. The program executed: **topCategories(5)** to find the **top 5 spending categories**.
5. The execution time was measured using **System.nanoTime()**.
6. We tested **three different input sizes**:

```java
testTopCategories( N: 10);        // small
testTopCategories( N: 1000);      // medium
testTopCategories( N: 100000);    // large
```

🎓 Lessons Learned & Structure

What I learned from this project

HashMap is fast: It helps store data using key–value pairs and makes searching for information very quick.

HashSet keeps data unique: It automatically removes duplicates, so the same value cannot appear twice.

Data structures make coding easier: Using structures like HashMap and HashSet helps organize data better and makes the program easier to understand.

Cleaner code: Methods like map.get() and set.contains() reduce the need for many loops and make the code shorter.

Error handling is important: Using try { } catch() helps prevent the program from crashing when something goes wrong.

Better understanding of Java collections: I learned how to work with collections and how they help manage data efficiently.

Problem solving: This project helped me think about how to choose the right data structure for different tasks.

Debugging skills: I practiced finding and fixing errors in my code.